Pagination, Sorting, Search, Swagger (Days 77-80)

1. Pagination + Sorting (Days 77-78)

Every list endpoint today returns a full `List<T>` — fine at test-data scale, wrong at real scale. Pattern to apply to each (start with `TransactionController.getTransactionHistory`, the one most likely to grow large):

Repository — just change the return/param type, Spring Data handles the rest:

```
Page<Transaction> findBySourceAccountIdOrDestinationAccountId(
        Long sourceAccountId, Long destinationAccountId, Pageable pageable);
```

Service:

```
public Page<TransactionSummaryResponse> getTransactionHistory(Long accountId, Pageable pageable) {
    Account account = accountRepository.findById(accountId)
            .orElseThrow(() -> new ResourceNotFoundException("Account not found with id:" + accountId));
    return transactionRepository
            .findBySourceAccountIdOrDestinationAccountId(accountId, accountId, pageable)
            .map(transactionMapper::toSummaryResponse);
}
```

Controller:

```
@GetMapping("/account/{accountId}/history")
public ResponseEntity<Page<TransactionSummaryResponse>> getTransactionHistory(
        @PathVariable Long accountId,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable)
    return ResponseEntity.ok(transactionService.getTransactionHistory(accountId, pageable));
}
```

Sorting comes free with `Pageable` — a client can call `?page=0&size=10&sort=amount,desc` with zero extra code on your side.

Test: `curl "http://localhost:8080/api/v1/transactions/account/1/history?page=0&size=2"` — response has `content`, `totalElements`, `totalPages`, `number` fields. Create 5+ transactions first so pagination is actually visible.

2. Search (Day 79)

Simplest version that covers real use cases — derived queries, no need for Specification/Querydsl machinery for this project's scale:

```
// CustomerRepository
List<Customer> findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCase(
        String firstName, String lastName);
```

```
// CustomerController
@GetMapping("/search")
public ResponseEntity<List<CustomerSummaryResponse>> search(@RequestParam String name) {
    return ResponseEntity.ok(customerService.searchByName(name));
}
```

Test: `curl "http://localhost:8080/api/v1/customers/search?name=raj"` — confirm it matches both first and last name, case-insensitively.

3. Swagger / OpenAPI (Day 80)

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
</dependency>
```

No other config needed for a working default. Optionally annotate for nicer docs:

```
@Tag(name = "Loans", description = "Loan application and approval")
@RestController
@RequestMapping("/api/v1/loans")
public class LoanController { ... }
```

Important: `SecurityConfig` currently only permits `/auth/**` — Swagger's UI and spec live at `/swagger-ui.html` and `/v3/api-docs`, which will otherwise require a JWT just to *view the docs page*. Add them to the permitted list:

```
.authorizeHttpRequests(auth -> auth
        .requestMatchers("/auth/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
        .anyRequest().authenticated())
```

Test: open `http://localhost:8080/swagger-ui.html` in a browser — every controller should be listed with its endpoints, request/response shapes, and a "Try it out" button that actually works (paste a JWT into the Authorize button first for anything beyond `/auth/**`).